



Roteiro de Aula - Insertion Sort

1. Definição

Insertion Sort, ou ordenação por inserção, é um algoritmo de ordenação por comparação simples e intuitivo.

A ideia é simples: a cada fase, o algoritmo seleciona o elemento atual (chamado de 'eleito') da parte não ordenada e o insere na sua posição correta dentro da sub-sequência à sua esquerda que já se encontra ordenada, deslocando os elementos maiores para a direita.

1. A parte à esquerda do vetor (sub-sequência V_0) permanece sempre ordenada.
2. A cada fase i , o item eleito é inserido em sua posição final relativa dentro de V_0 .
3. O algoritmo usa pouca memória extra: apenas variáveis auxiliares para o eleito e índices (algoritmo in-place e estável).

Pergunta: Como o Insertion Sort abre espaço na parte ordenada para posicionar o elemento eleito?

Ele pega o eleito e desloca enquanto o eleito for menor que o anterior. Depois coloca o eleito na posição correta

2. Antes de programar: simule uma execução

Use o vetor abaixo e complete as fases manualmente.

Vetor inicial: [34, 17, 68, 29, 50, 47]

Fase	Sub-sequência Ordenada (V_0)	Elemento Eleito	Vetor após a inserção
$i = 1$	34	17	[17,34, 68, 29, 50, 47]
$i = 2$	17,34	68	[17,34, 68, 29, 50, 47]
$i = 3$	17,34,68	29	[17,34, 29, 68, 50, 47]
$i = 4$	17,29,34,68	50	[17,34, 29, 50, 68, 47]
$i = 5$	17,29,34,50,68	47	[17,34, 29, 47, 50, 68]



Pergunta: Quantas comparações e movimentações ocorrem no pior caso do Insertion Sort (vetor em ordem decrescente)?

3. Complete o código da classe InsertionSort

Crie a classe InsertionSort. Complete as lacunas do código. Repare que a classe é genérica: ela pode ordenar Integer, String ou outros tipos, desde que exista um Comparator.

```
import java.util.Comparator;

public class InsertionSort<T extends Comparable<T>> {
    private long contaComparacoes;
    private long contaDeslocamentos;

    public long getContaComparacoes() {
        return contaComparacoes;
    }

    public long getContaTrocas() {
        return contaDeslocamentos;
    }

    private void insert(T[] v, int i, Comparator<T> comparador) {
        T eleito = v[i];
        int indiceComparacao = i - 1;

        while (indiceComparacao >= 0 && comparador.compare(eleito,
v[indiceComparacao]) < 0) {
            contaDeslocamentos++;
            contaComparacoes++;
            v[indiceComparacao + 1] = v[indiceComparacao];
            indiceComparacao--;
        }
    }
}
```

```
    }

    contaComparacoes++;

    v[indiceComparacao + 1] = eleito;
}

public T[] sort(T[] array, Comparator<T> comparador) {
    int n = array.length;
    this.contaComparacoes = 0;
    this.contaDeslocamentos = 0;

    for (int i = 1; i < n; i++) {
        insert(array, i, comparador);
    }
    return array;
}

public T[] sort(T[] array) {
    return sort(array, (a, b) -> ((Comparable<T>) a).compareTo(b));
}}
```



Pergunta: Onde o método insert realiza a movimentação dos elementos e a contagem das comparações e trocas? Explique.

```
v[indiceComparacao + 1] = v[indiceComparacao];
```

ele desloca a posição para a direita abrindo espaço para o eleito



Pergunta: Por que o laço principal do método sort inicia em $i = 1$ ao invés de $i = 0$?

eu considero o 0 parte ordenada e a partir do 1 a parte desordenada

4. Implementando o main

Agora crie uma classe main para executar Insertion Sort, Selection Sort e Bubble Sort no mesmo vetor inicial e comparar tempo, trocas e comparações.

Utilize o método para gerar números aleatórios

```
public static Integer[] gerarVetorAleatorio(int tamanho) {  
    Integer[] vetor = new Integer[tamanho];  
    for (int i = 0; i < tamanho; i++) {  
        vetor[i] = (int) (Math.random() * 100);  
    }  
    return vetor;  
}
```



Apresente o código abaixo.

```
public class App {  
  
    public static Integer[] vetorDeNumeros() {  
  
        return new Integer[]{  
  
            88, 55, 44, 12, 10, 8  
  
        };  
    }  
  
    public static Integer[] gerarVetorAleatorio(int tamanho) {  
  
        Integer[] vetor = new Integer[tamanho];  
  
        for (int i = 0; i < tamanho; i++) {  
  
            vetor[i] = (int) (Math.random() * 100);  
  
        }  
  
        return vetor;  
    }  
  
    public static void main(String[] args) throws Exception {  
  
        int n = 150000;  
  
  
        long inicio, fim, tempo;
```

```
Integer[] numerosSelectionSort = gerarVetorAleatorio(n); //vetorDeNumeros();

Integer[] numerosBubbleSort = numerosSelectionSort.clone();

Integer[] numerosInsertionSort = numerosSelectionSort.clone();

//System.out.println("Vetor gerado:"+ Arrays.toString(numeros));

SelectionSort selectionSort = new SelectionSort<>();

inicio = System.currentTimeMillis();

selectionSort.sort(numerosSelectionSort);

fim = System.currentTimeMillis();

tempo = fim - inicio;

//System.out.println("Vetor ordenado:"+ Arrays.toString(numeros));

System.out.println("\nComparacoes
Selection:"+selectionSort.getContaComparacoes());

System.out.println("Trocas Selection:"+selectionSort.getContaTrocas());

System.out.println("Tempo(ms) do Selection:"+tempo);

// Bubble Sort

BubbleSort bubbleSort = new BubbleSort<>();

inicio = System.currentTimeMillis();

bubbleSort.sort(numerosBubbleSort);

fim = System.currentTimeMillis();

tempo = fim - inicio;
```

```
        System.out.println("\nComparacoes Bubble Sort: "
+bubbleSort.getContaComparacoes());

        System.out.println("Trocas Bubble Sort: " +bubbleSort.getContaTrocas());

        System.out.println("Tempo(ms) do BubbleSort:"+tempo);

        // Insertion Sort

        InsertionSort insertionSort = new InsertionSort<>();

        inicio = System.currentTimeMillis();

        insertionSort.sort(numerosInsertionSort);

        fim = System.currentTimeMillis();

        tempo = fim - inicio;

        System.out.println("\nComparacoes ISort: "
+insertionSort.getContaComparacoes());

        System.out.println("Deslocamentos ISort: "
+insertionSort.getContaDeslocamentos());

        System.out.println("Tempo(ms) do InsertionSort:"+tempo);

    }

}
```

5. Comparando resultados

Execute o programa e preencha a tabela com os valores exibidos no console.

Algoritmo	N	Comparações	Trocas / Deslocamentos	Tempo de execução (milis)
Insertion Sort	50000	615694956	615644964	3389
Selection Sort	50000	1249975000	49999	13504
Bubble Sort	50000	1249975000	615644964	4211
Insertion Sort	150000	5575434718	5575284725	39302
Selection Sort	150000	11249925000	149999	29073
Bubble Sort	150000	11249925000	5575284725	125885



Pergunta: Qual algoritmo se comportou melhor em tempo e movimentações de dados? O resultado do Insertion Sort confirma o que foi visto na teoria?

O insertion com 50000 e selection com 150000 elementos. O Insertion Sort tende a ter um bom desempenho em situações favoráveis, mas, como os testes foram executados em Java, o tempo também pode sofrer influência da JVM e de outros fatores de execução. Portanto, o resultado confirma parcialmente o que foi visto na teoria.



Pergunta: Agora rode os algoritmos em C e depois em o Bubble Sort em python, o que observou? Explique.

Muda todo o comportamento. Mesmo com o mesmo algoritmo a linguagem influencia muito, em C o Selection sempre vai ser mais rápido.

Fique ligado, não se esqueça das premissas abaixo

1. Insertion Sort realiza poucas comparações e movimentações em vetores quase ordenados, atingindo complexidade $O(n)$ no melhor caso.
2. Ao contrário do Selection Sort (que faz $O(n^2)$ comparações sempre), o Insertion Sort pode interromper as comparações da fase assim que encontra a posição correta do eleito.
3. Insertion Sort é um algoritmo estável, preservando a ordem relativa original de elementos com valores iguais.
4. Embora Insertion Sort, Selection Sort e Bubble Sort possuam complexidade $O(n^2)$ no pior caso, o tempo de execução e a quantidade de movimentações variam significativamente na prática.